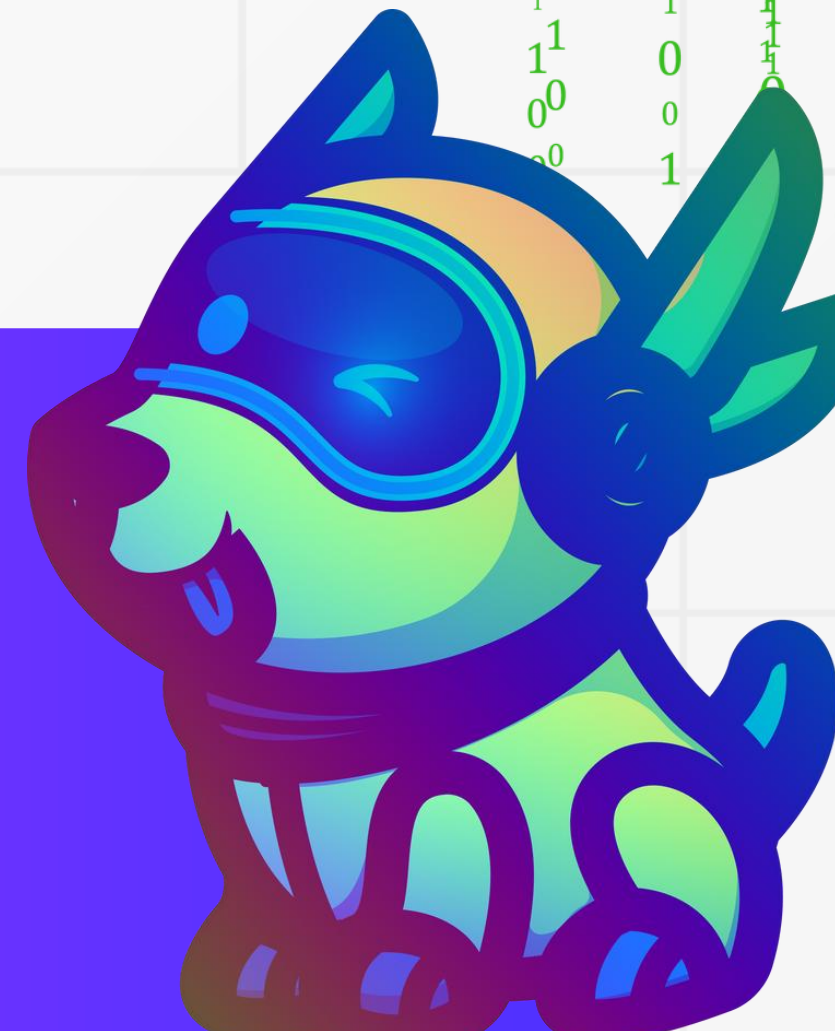


GenAI & Prompt Engineering



Generative AI (GenAI)

GenAI is an artificial intelligence (AI) model that can

- **create new content** (text, images, music, videos, code)
- based on patterns and structures **learned**
(via neural networks)
- from large **datasets**
- that resembles human-created content.

GenAI vs. LLMs

Although people use “Generative AI (GenAI)” and “Large Language Models (LLMs)” interchangeably, GenAI is actually an umbrella concept whereas LLMs are one specific type of GenAI.

All LLMs are GenAI, but not all GenAI are LLMs.

- **ChatGPT 5.2** – this is a general large language model
- **OpenAI Codex** – this is a LLM specialized for code. It was trained heavily on programming languages and designed to generate, explain, and modify code.
- **DALL.E** – this is GenAI, but not an LLM because it generates images, not text.

Generative AI (GenAI) for Coding Activities

GenAI can be used for the following code activities:

- Debugging or troubleshooting code
- Understanding code
- Making code more efficient
- Generating documentation / comments
- Generating pseudocode
- Generating code in a specific language
- Suggesting how data should be organized for a given problem (suggesting type of data structure)
- Suggesting algorithm to use

Reasons **to Trust** GenAI for Coding Activities

Using GenAI to help in code activities can:

- allow you to be more marketable and **ready for industry expectations**.
- help you **write more efficient code**.
- help you **create programs faster** (generating comments, create simple functions quickly, etc.)
- help you **troubleshoot buggy code**.
- help you **test your code** by producing test cases.

Reasons **Not to Trust** GenAI for Coding Activities

Content generated by GenAI:

- can be **wrong** (hallucinations, based off incorrect data)
- can be **inconsistent** even with same prompt
- can **ignore directions or constraints**
(example: "don't provide the solution" or "don't use arrays")
- Can use **inefficient code** or produce code using **bad programming practices**.
- can be **unethical** (not allowed by course, plagiarized content, not sustainable, etc.)
- Can **use data without permission** (security issue)

Calibrated Trust in GenAI for Coding Activities

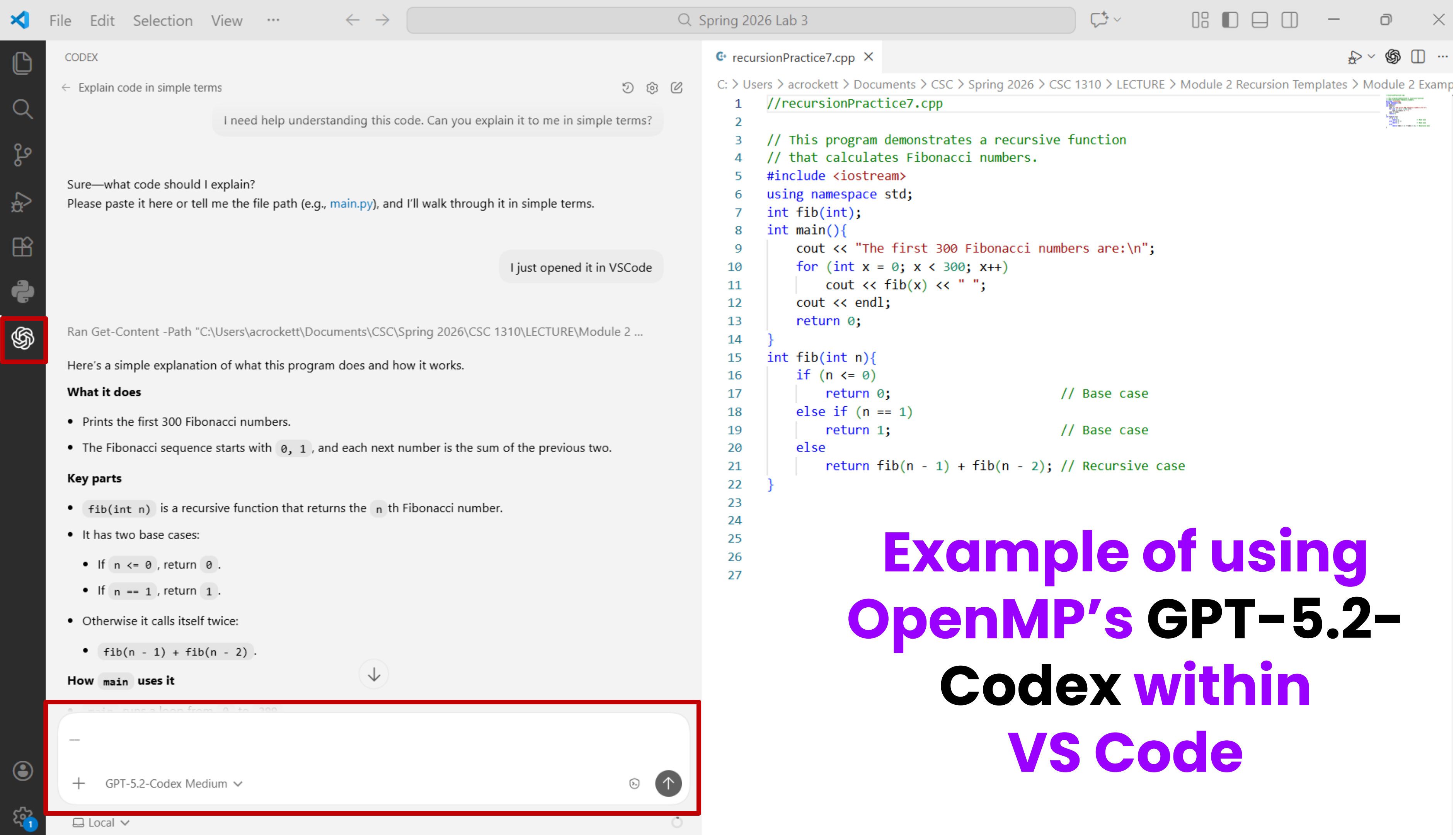
(having calibrated trust is the goal)

The more you use GenAI for diverse coding activities, the better you will be able to:

- prompt and converse with GenAI (prompt engineering)
- understand its limitations
- understand when to trust responses (and when not to)
- understand what types of content and activities it is more accurate with (what is GenAI most useful for)
- know when it is ethical to use and when it is not
- use GenAI secure practices – keeping people's data confidential

Top Two LLMs for Coding

- **OpenAI**
 - ChatGPT (5.2 latest model)
 - Codex (only for paid Plus subscription – currently \$20/month)
- **Anthropic Claude**
 - Claude (Sonnet 4.5 is latest free model, Opus 4.5 is paid, advanced version)
 - Claude Code (also only for paid “Pro” subscription or higher – currently \$17/month)



DEMO 1: Sometimes GenAI can be Unintelligent

Look at what happened when I prompted “**show me a seahorse emoji**”

On OpenAI ChatGPT 5.0: <https://chatgpt.com/share/6915507b-931c-8003-bdc9-f62adac858d6>

And on Claude Sonnet 4.5: <https://claude.ai/share/e332fbe1-52d0-40ae-9161-1f2ab7284596>

DEMO 2: Asking to explain a compile error

The Program:

```
demo2.cpp
1  #include <iostream>
2  int main() {
3      int num == 4;
4      while(num)
5          cout << num is true! >> endl;
6          num--;
7      return 0;
8  }
```

Errors:

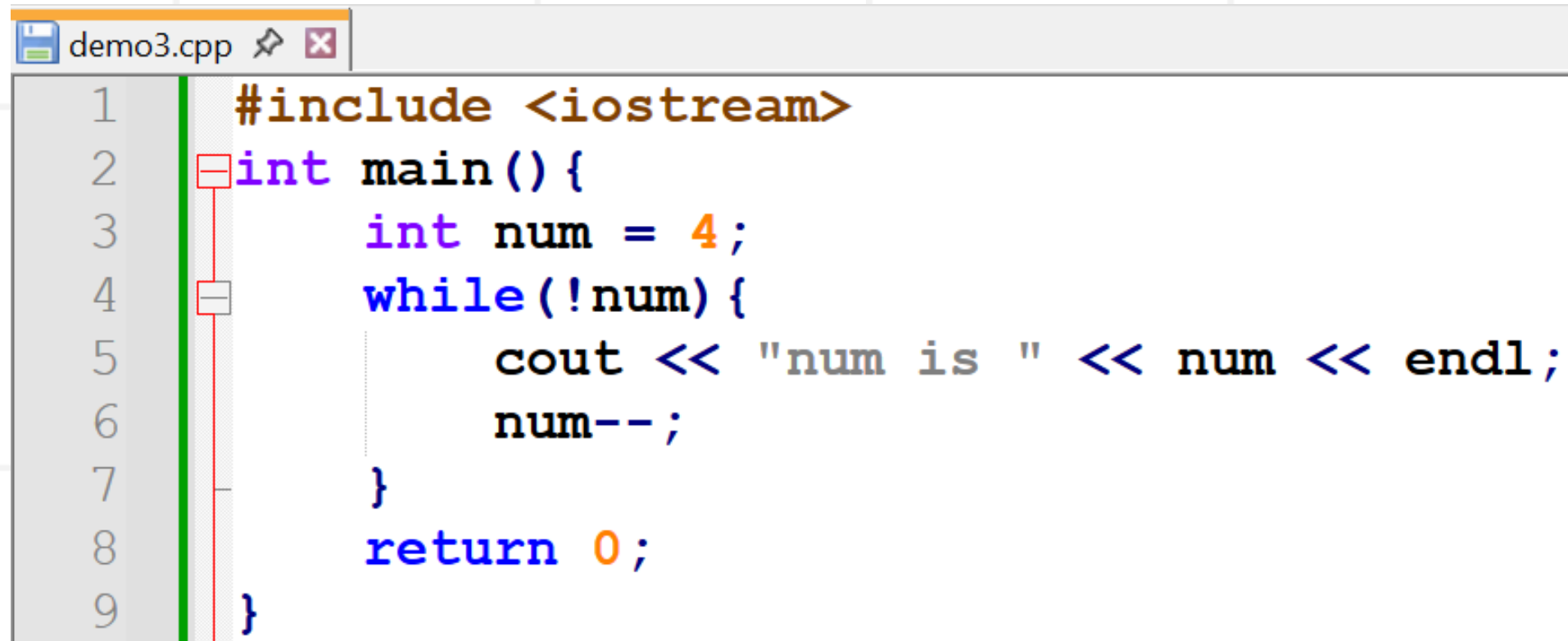
ChatGPT: <https://chatgpt.com/share/69155657-2b08-8003-b8a6-55eb01c4a3c2>

Claude: <https://claude.ai/share/0eb3c6bb-0fcf-4d9d-94f9-951f5fa19be2>

```
demo2.cpp: In function 'int main()':
demo2.cpp:3:17: error: expected initializer before '==' token
    3 |         int num == 4;
      |                ^~
demo2.cpp:4:15: error: 'num' was not declared in this scope; did you mean 'enum'?
    4 |         while(num)
      |                ^~~
      |                enum
demo2.cpp:5:17: error: 'cout' was not declared in this scope; did you mean 'std::cout'?
    5 |             cout << num is true! >> endl;
      |             ^~~~
      |             std::cout
```

DEMO 3: Asking about Runtime Behavior

The Program:



```
1  #include <iostream>
2  int main() {
3      int num = 4;
4      while(!num) {
5          cout << "num is " << num << endl;
6          num--;
7      }
8      return 0;
9  }
```

What if I didn't understand why nothing is being printed out.

I could just ask...

ChatGPT: <https://chatgpt.com/share/69155788-1714-8003-b163-ffdb7fa6e122>

Claude: <https://claude.ai/share/a0c3a7ab-db8d-4cbb-9e66-bd665c3836f8>

DEMO 4: Finding Logic Errors

The Program:

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      double grade;
6
7      cout << "\nWhat is your final grade in CSC 1300? ";
8      cin >> grade;
9
10     if(grade > 70)
11         cout << "Yay! You can move on to CSC 1310!\n";
12     if(grade < 70)
13         cout << "Sorry, you must make a C or better in CSC 1300 to take CSC 1310.\n";
14
15     return 0;
16 }
```

You can paste your code and ask if there are any logic errors.

This can help with testing your code & finding bugs in your logic.

ChatGPT: <https://chatgpt.com/share/691559f7-c32c-8003-8c26-fb75d5dca756>

Claude: <https://claude.ai/share/5d73c42b-c026-4ff7-8b06-abf0ad26b7e7>

Asking for Explanation in Simple Terms

🐼 🐷 Bonus Good Practice Note

Using an `else` makes the logic cleaner because the two outcomes are mutually exclusive.

Let me know if you'd like to expand this to handle invalid input (negative numbers, letters, etc.)!



Explain to me in very simple terms why using an `else` makes the logic cleaner.

Basics of a Good Prompt – Iterate

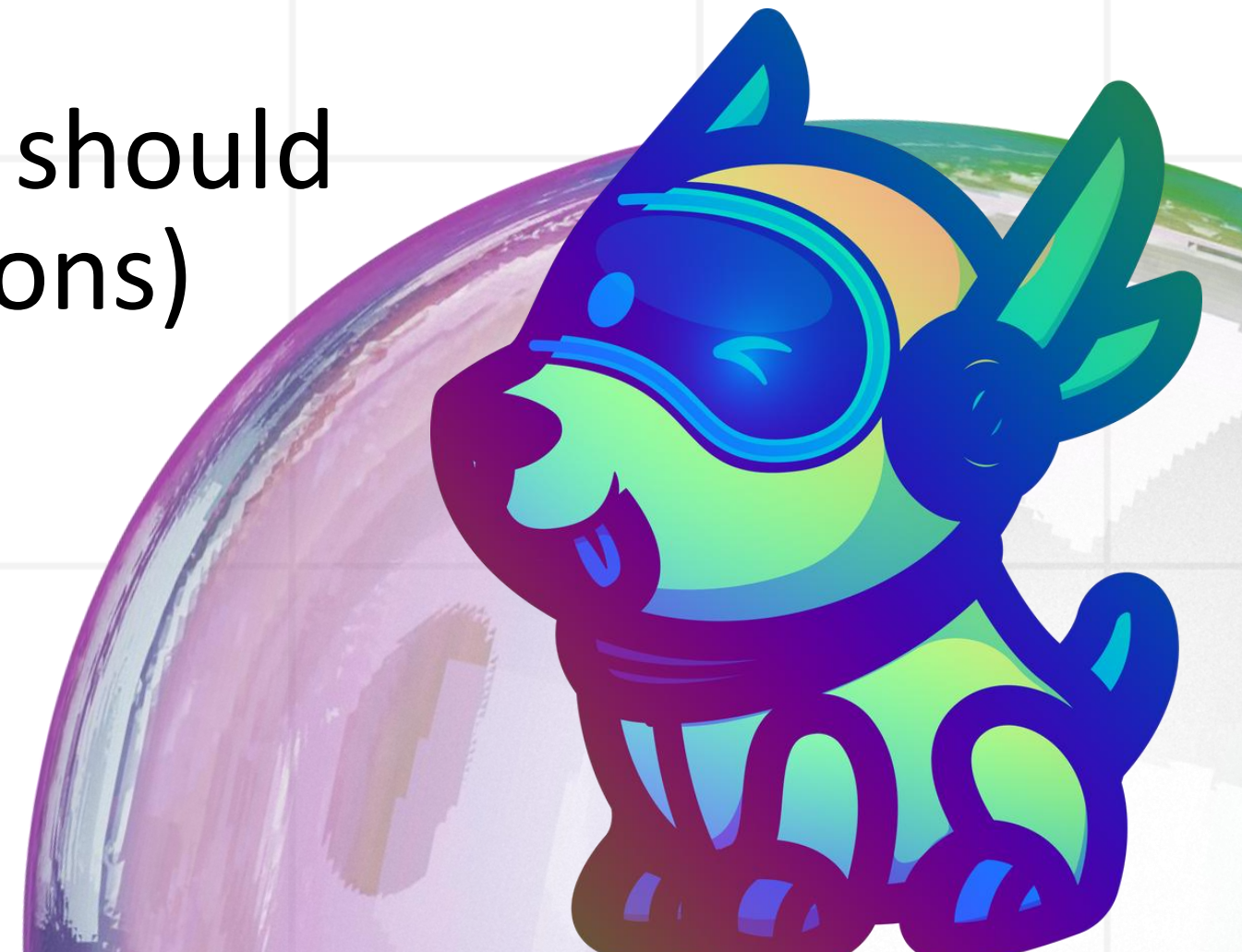
- Think of prompting as **having a conversation**, not a one-shot question & response.
- **Iterate**. Refine the prompt based on the AI's response.



RTRI Blueprint for Prompt Engineering

(from University of Sydney)

- **Role:** Who the AI should act as
- **Task:** What you want help with
- **Requirements:** Constraints or rules
- **Iteration / Interpretation:** How the response should support learning (explanations, checks, revisions)



RTRI Blueprint Demo

Weak (non-RTRI) Prompt:

“Fix my C++ loop.”
(with uploaded code)

RTRI-Based Prompt:

Role

→ “Act as a CS2 teaching assistant.

Task

→ Help me identify the base case error in my recursive function.

Requirements

→ Do NOT rewrite my entire program. Explain the logic error & suggest what I should check.

Iteration/Interpretation

→ After explaining, ask me one question to verify that I understand the issue. Here is my code:”
(with uploaded code)

END!

